

Storage Chiave-Valore Replicato e Consistente in Go

Federico Cola

Corso di Sistemi Distribuiti e Cloud Computing

Università degli Studi di Roma “Tor Vergata”

Email: federicola00@gmail.com

Sommario—Questo lavoro presenta un key-value store distribuito, replicato e fortemente consistente, costruito in Go attorno a un’implementazione da zero dell’algoritmo di consenso Raft. Il sistema è composto da tre tipologie di nodo — *consensus node* stateful, *client proxy* stateless e servizio di *snapshot & backup* stateless — che comunicano esclusivamente via gRPC e Protocol Buffers, e integra due pattern architetturali, Service Discovery e Circuit Breaker, per la scoperta dinamica del leader e l’isolamento dei nodi non raggiungibili. Il deployment è containerizzato con Docker Compose e verificato su una vera istanza AWS EC2. Il sistema è stato valutato con due esperimenti: scalabilità (tempo di risposta delle RPC al variare del numero di nodi, da 3 a 20) e fault-tolerance (tempo di rielezione e convergenza dopo il crash del leader). I risultati sono coerenti con la teoria di Raft — latenza di lettura indipendente dal numero di nodi, latenza di scrittura che cresce solo con quorum di maggioranza numerosi, tempi di rielezione in linea con i timeout configurati — e il percorso sperimentale ha messo in luce l’importanza di un ambiente di misura isolato per benchmark di latenza affidabili.

I. INTRODUCTION

Questo lavoro presenta la progettazione e l’implementazione di un key-value store distribuito, replicato e fortemente consistente, sviluppato nell’ambito del progetto B5 del corso di Sistemi Distribuiti e Cloud Computing. Il sistema non si appoggia a nessun database centrale: la consistenza dello stato tra le repliche è garantita interamente dall’algoritmo di consenso distribuito **Raft** [1], implementato da zero.

L’architettura è composta da tre tipologie di nodo con responsabilità distinte: un cluster di *consensus node* stateful, che eseguono l’algoritmo di consenso e mantengono lo stato replicato; un *client proxy* stateless, che instrada le richieste dei client verso il nodo Leader corrente scoprendolo dinamicamente; e un servizio di *snapshot & backup* stateless, che effettua periodicamente il checkpoint dello stato applicativo. La comunicazione interna tra i nodi avviene tramite RPC su gRPC e Protocol Buffers, e il sistema integra due pattern architetturali — Service Discovery e Circuit Breaker — per gestire rispettivamente la scoperta dinamica del Leader e l’isolamento dei nodi non raggiungibili. Il deployment è containerizzato con Docker Compose ed è stato verificato su un’istanza reale AWS EC2.

Lo sviluppo è stato affiancato dall’uso di strumenti di intelligenza artificiale, impiegati principalmente come supporto alla scrittura delle sezioni di codice, mentre le scelte progettuali, la verifica del comportamento del sistema e l’interpretazione dei

risultati sperimentali sono state guidate direttamente dall’autore. L’intero percorso di lavoro è tracciato attraverso un sistema di versionamento (Git/GitHub), con commit incrementali e descrittivi per ciascuna fase; il codice sorgente completo e commentato è disponibile pubblicamente all’indirizzo <https://github.com/Cellu83/sdcc-Progetto-b5-kvstor>.

Il resto del report è organizzato come segue: la Sezione II richiama i concetti di consenso distribuito necessari a comprendere le scelte progettuali; la Sezione III descrive l’architettura del sistema; la Sezione IV entra nel merito delle scelte implementative più significative; la Sezione V presenta i risultati sperimentali di scalabilità e fault-tolerance; la Sezione VI discute tali risultati e i limiti noti del sistema; la Sezione VII chiude il lavoro.

II. BACKGROUND

In un sistema distribuito che replica il proprio stato su più nodi per tollerare guasti, il problema centrale è far sì che tutte le repliche concordino sulla stessa sequenza di operazioni, anche in presenza di crash di singoli nodi o di ritardi di rete. Risolvere questo problema senza un coordinatore centrale (che sarebbe esso stesso un singolo punto di fallimento) è l’obiettivo degli algoritmi di *consenso distribuito*.

Tra le soluzioni note, Paxos [2] è storicamente la più citata, ma è notoriamente difficile da adattare e da implementare correttamente. Per questo progetto si è scelto **Raft** [1], progettato esplicitamente per essere semplice senza sacrificare le garanzie di correttezza di Paxos, scomponendo il problema in tre sotto-problemi relativamente indipendenti: elezione del leader, replicazione del log e sicurezza.

I concetti chiave di Raft rilevanti per le sezioni successive sono:

- **Term**: un contatore logico monotono crescente che identifica un intervallo di tempo con al più un Leader. Ogni messaggio tra nodi porta il term del mittente, e un term più recente prevale sempre su uno più vecchio.
- **Ruoli**: ogni nodo è in un dato momento Follower, Candidate o Leader. Un Follower che non riceve heartbeat dal Leader diventa, entro un timeout randomizzato, un Candidate e avvia una nuova elezione; vince chi ottiene il voto della maggioranza dei nodi.
- **Log replication**: solo il Leader accetta scritture dai client; le aggiunge al proprio log e le replica ai Follower. Un’entry è *committata*, e quindi applicabile alla state machine,

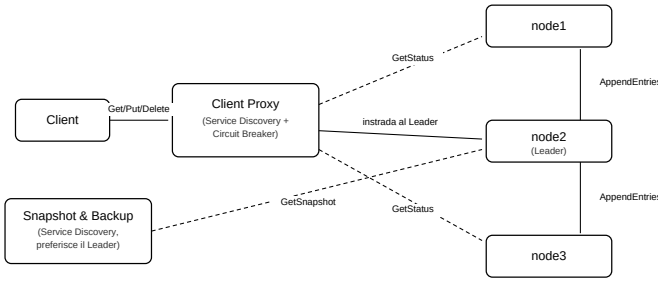


Figura 1. Architettura del sistema: tre tipologie di nodo comunicanti via gRPC. Freccie continue: RPC applicative/di replica. Freccie tratteggiate: RPC di sola lettura usate per la Service Discovery e per lo Snapshot & Backup service.

solo dopo essere stata replicata su una maggioranza dei nodi.

- **Election restriction:** un candidato può ottenere un voto solo se il proprio log è aggiornato almeno quanto quello di chi vota — condizione che impedisce di eleggere un Leader che perderebbe entry già committate.
- **La regola di sicurezza §5.4.2:** un leader può committare per semplice conteggio di maggioranza solo le entry del proprio term corrente, mai quelle di un term precedente (anche se già su una maggioranza); deve aspettare che si comitti insieme a (almeno) un'entry del proprio term corrente. È una regola sottile ma essenziale per la correttezza, discussa più avanti (Sezioni IV e VI) anche alla luce di un'osservazione empirica avvenuta durante il deployment su cloud.

Infine, la comunicazione tra i nodi richiede un meccanismo di RPC efficiente e tipizzato: il sistema usa gRPC con interfacce definite in Protocol Buffers, sia per le RPC interne dell'algoritmo di consenso (RequestVote, AppendEntries) sia per l'API applicativa esposta ai client (Get/Put/Delete).

III. SOLUTION DESIGN

Il sistema è composto da tre tipologie di nodo, ciascuna un binario Go indipendente, che comunicano esclusivamente via RPC gRPC/Protocol Buffers (Figura 1).

Consensus node (stateful): eseguono l'algoritmo Raft e mantengono lo stato replicato — sia il log persistito (CurrentTerm, VotedFor, le entry) sia la state machine chiave-valore in memoria. Espongono RaftService (RequestVote, AppendEntries, e le RPC di sola lettura GetStatus e GetSnapshot) e, solo se correntemente Leader, rispondono anche alle richieste applicative di KVStoreService.

Client proxy (stateless): unico punto di contatto per i client esterni. Non mantiene stato applicativo proprio; il suo unico compito è instradare in modo trasparente ogni richiesta verso il Leader corrente, riprovando verso un altro nodo se quello contattato non lo è (più), realizza questo tramite i due pattern architetturali richiesti dalla traccia.

Snapshot & backup service (stateless): interroga periodicamente un consensus node raggiungibile per ottenere un'istantanea dello stato applicativo corrente, e la scrive su un checkpoint esterno persistente — un backup indipendente dal ciclo di vita di ogni singolo nodo.

A. Service Discovery

Nessun componente del sistema conosce, staticamente, l'indirizzo dell'attuale Leader — perché può cambiare in qualunque momento, per elezione o per crash. Il pattern Service Discovery (`internal/discovery`) risolve questo interrogando periodicamente i nodi noti tramite la RPC di sola lettura `GetStatus`, che restituisce chi il nodo interrogato ritiene sia il Leader corrente. Sia il Client proxy sia lo Snapshot & backup service riusano lo stesso meccanismo: partono da una lista statica di nodi seed (nota da configurazione) e mantengono da lì in poi una vista sempre aggiornata di peer e Leader, senza mai avere un indirizzo fisso scritto nel codice.

B. Circuit Breaker

Il Circuit Breaker (`internal/proxy`) applicato dal Client proxy a ogni indirizzo tiene traccia dei fallimenti consecutivi verso quell'indirizzo con tre stati: *closed* (funzionamento normale), *open* (troppi fallimenti consecutivi: le chiamate verso quell'indirizzo vengono scartate immediatamente, senza nemmeno provare la rete, per un periodo di raffreddamento), *half-open* (raffreddamento scaduto: una sola chiamata di prova per verificare se il nodo è tornato disponibile). Combinato con la Service Discovery, questo permette al proxy di isolare rapidamente un nodo caduto e ridirigersi verso il Leader corrente senza pagare ripetutamente il costo dei timeout di rete verso nodi che sa già irraggiungibili.

IV. SOLUTION DETAILS

A. Configurazione esterna, nessun valore hard-coded

All'interno del progetto nessun parametro operativo (ID del nodo, indirizzi dei peer, porte, timeout di elezione, intervallo di heartbeat, livello di log) vive nel codice: tutto passa da un file YAML caricato tramite struct tipizzate (`internal/config`), che vengono validate al momento del caricamento. Questa scelta, fa in modo che in Docker Compose si possano semplicemente passare configurazioni diverse (Sezione IV-F), e che lo strumento usato per generare i cluster di N nodi per gli esperimenti di scalabilità (Sezione V) riusi direttamente le stesse struct invece di modificare il file YAML manualmente.

B. Persistenza atomica su disco

I tre pezzi di stato che Raft impone di rendere durevoli — CurrentTerm, VotedFor, e il log delle entry — sono scritti su disco (`internal/raftlog`) con uno schema di scrittura atomica: prima su un file temporaneo, poi con una `rename` sopra il file definitivo. Questo garantisce che un crash del processo a metà scrittura non lasci mai uno stato corrotto o parziale: al riavvio il nodo trova o lo stato precedente (se il crash è avvenuto prima della `rename`) o quello nuovo completo

(se dopo), mai una via di mezzo. Lo stesso schema è riusato, senza modifiche concettuali, dal servizio Snapshot per scrivere i propri checkpoint esterni.

Da notare, e rilevante per i risultati della Sezione V: `commitIndex` e `lastApplied` sono deliberatamente *volatili* e si azzerano a ogni riavvio del processo: è così che il paper di Raft li definisce, e il sistema si comporta correttamente anche in questo caso limite grazie alla regola di sicurezza §5.4.2 (Sezione II).

C. Replicazione del log: `nextIndex` e `matchIndex`

Per ogni follower, il Leader mantiene due valori volatili che si ricostruiscono da zero a ogni nuova elezione, `nextIndex[id]` e `matchIndex[id]`, rispettivamente il prossimo indice di log da inviare a quel follower e l'indice più alto che si sa già replicato su di lui.

La replica verso ogni peer è innescata sia periodicamente, a ogni tick dell'heartbeat (50ms), sia immediatamente dopo una `Propose`, così una nuova scrittura non deve attendere il prossimo tick per iniziare a essere replicata. Per ciascun follower il Leader invia, in una singola RPC `AppendEntries`, `PrevLogIndex/PrevLogTerm` (il punto del log da cui il follower dovrebbe già essere allineato) seguiti da *tutte* le entry dal proprio `nextIndex` fino alla fine del log — non una entry alla volta.

L'aggiornamento di `nextIndex/matchIndex` avviene in modo sincrono, non appena arriva la risposta a quella RPC:

- se il follower accetta (`Success = true`, perché la sua entry a `PrevLogIndex` coincide con quella del Leader), `matchIndex[id]` viene alzato all'indice più alto appena confermato — con una guardia che ne impedisce la retrocessione, dato che più chiamate verso lo stesso follower possono essere in volo contemporaneamente (una dal ciclo di heartbeat, una da una `Propose`) e le risposte possono arrivare in un ordine diverso da come sono partite. L'avanzamento di `matchIndex` innesca subito, nella stessa sezione critica, il tentativo di far avanzare il `commitIndex` del cluster controllando la aggiornata dei nodi (Sezione II).
- se il follower rifiuta (`Success = false`, conflitto di log a `PrevLogIndex`), `nextIndex[id]` viene decrementato di uno, e si riproverà al giro successivo con un punto di partenza più indietro.

Un dettaglio rilevante per la velocità di recupero di un follower rimasto indietro: il passo indietro (`nextIndex--`) avviene una posizione alla volta finché non si trova il punto d'accordo col Leader, ma una volta trovato, il passo in avanti recupera *tutto* il ritardo accumulato in un'unica RPC (tutte le entry mancanti insieme), non un'entry per round.

D. Backoff di riconnessione gRPC vs. timeout di elezione

Testando dal vivo scenari di crash e riavvio ripetuti (uccidendo e riavviando nodi in varie combinazioni tramite Docker), è emerso un pattern sistematico: un nodo riavviato avviava *sempre* una nuova elezione, non riconosceva mai in silenzio il Leader già attivo — anche quando rientrava in

poche centinaia di millisecondi. La causa non era casuale ma strutturale: `internal/raft/client.go` riusa la stessa connessione gRPC verso ogni peer (`Client.connFor`), ed è corretto farlo per evitare di riaprire una connessione TCP a ogni RPC. Il problema è che gRPC, di default, applica un backoff di riconnessione con `BaseDelay` di 1 secondo — pensato per reti geografiche reali — quando una connessione cade. Quando un nodo veniva ucciso e poi riavviato, il Leader restava fermo nel proprio timer di backoff (>1s) prima di ritentare la connessione verso di lui, ben oltre il timeout di elezione del nodo riavviato (150–300ms): quest'ultimo scadeva sempre prima di ricevere un heartbeat, e si candidava — spesso vincendo, dato che il suo log restava comunque aggiornato quanto quello degli altri.

Corretto impostando un `BaseDelay` molto più breve (50ms) tramite `grpc.WithConnectParams` su ogni connessione verso i peer. Verificato con un test locale con processi nativi: un follower ucciso e riavviato torna a riconoscere il Leader esistente in meno di 150ms, senza elezione. Ripetendo la stessa prova in container Docker, il comportamento è invece risultato variabile — a volte il nodo riconosce il Leader, a volte si candida comunque. Il motivo non è più il backoff (ormai trascurabile), ma un limite fisico diverso: il tempo che Docker impiega a far ripartire davvero il container (processo, listener gRPC) prima che il Leader possa riconnettersi, che può da solo superare il timeout di elezione a seconda del carico della macchina — un vincolo della piattaforma di containerizzazione, non del codice applicativo.

E. Compattazione dello storico in checkpoint esterni

Il servizio Snapshot & backup soddisfa il requisito di "scaricare e compattare i log storici in file di checkpoint stabili": la RPC `GetSnapshot` non restituisce il log grezzo, ma lo stato corrente della state machine (`Store.Snapshot()`) — l'intera storia di operazioni `Put/Delete` già compattata nel suo risultato finale. Il servizio scarica questo stato e lo scrive su un checkpoint esterno persistente, in sola lettura verso i nodi, indipendente dal ciclo di vita di ognuno di essi.

Un'implementazione più tradizionale di Raft userebbe tipicamente lo snapshot anche per comprimere il log sui consensus node stessi (troncando le entry già incluse nello snapshot). In questo progetto si è scelto deliberatamente di *non* farlo: il servizio resta puramente esterno, senza mai modificare il log o gli indici di nessun consensus node. La motivazione è non rimettere in discussione la matematica degli indici (`commitIndex`, `matchIndex`, la regola §5.4.2) già verificata e mantenere il servizio funzionante maggiormente separato dal sistema del cluster.

F. Containerizzazione

Il deployment usa un solo Dockerfile generico multi-stadio, parametrizzato via `build-arg` (`SERVICE`), invece di tre Dockerfile quasi identici: uno stadio di build con l'intero toolchain Go, e uno stadio finale minimale che contiene solo il binario compilato. `docker-compose.yml` definisce 7 servizi (5 consensus node, proxy, snapshot) che riferiscono tutti lo stesso

Dockerfile con un `SERVICE` diverso; le configurazioni per l'ambiente Docker sono identiche a quelle locali tranne per gli indirizzi dei peer, che usano i nomi dei servizi Compose invece di `localhost` (risolti via il DNS interno della rete bridge di Compose). Containerizzare non ha richiesto nessuna modifica al codice applicativo — conseguenza diretta della configurazione esterna descritta sopra.

Il deployment è stato verificato su una vera istanza AWS EC2 (`t2.micro`, AWS Academy Learner Lab), non solo in locale (Figura 2): i 5 container avviati con `docker compose up -d`, l'elezione del leader osservata nei log del container stesso (con i peer risolti via DNS come `node1:50051`, non `localhost`), l'istanza raggiungibile dall'esterno sul suo IP pubblico, e la porta del proxy (50060) aperta nel Security Group per consentire chiamate reali da fuori l'istanza.

G. Limitazione nota: nessun cambio di membership dinamico

La lista dei peer di ogni nodo è statica, letta una volta sola dal file YAML all'avvio, e non cambia mai durante l'esecuzione: non esiste nessuna RPC per aggiungere un nodo a un cluster già in funzione. Un nodo *già noto* che crasha e riparte si riallinea automaticamente (il meccanismo di coerenza del log descritto sopra lo riporta in sincrono col Leader), ma un nodo *nuovo*, mai presente nella configurazione iniziale, non può unirsi a runtime: nessun Leader lo conterebbe per il quorum di maggioranza, e la Service Discovery non può scoprire un nodo che nessun altro nodo conosce già. È il cambio di membership dinamico che il paper di Raft tratta a parte (§6, joint consensus) — deliberatamente fuori scope per questo progetto, non richiesto dalla traccia.

V. RESULTS

Entrambi gli esperimenti sono stati eseguiti su un'istanza AWS EC2 (`t2.micro`) dell'infrastruttura di deployment, non sul portatile di sviluppo: un primo tentativo in locale aveva prodotto misure rumorose e non monotone (Sezione VI), e ripetere gli stessi esperimenti su un'istanza dedicata — pur con hardware oggettivamente più debole — ha dato un segnale molto più pulito, perché quella macchina non esegue nient'altro in concorrenza con l'esperimento.

A. Scalabilità: latenza RPC al variare di N

Lo strumento `bench-client` misura la latenza di `Put` e `Get` in sequenza (una RPC alla volta) contro il proxy, per $N = 3, 5, 9, 20$ nodi consensus. Ogni misura aggrega 5 ripetizioni indipendenti da 50 operazioni ciascuna, dopo 5 operazioni di riscaldamento scartate per stabilizzare le connessioni gRPC. La Tabella I riporta media, mediana (p50) e 95° percentile (p95) in millisecondi.

Tabella I
LATENZA RPC (MS) AL VARIARE DEL NUMERO DI NODI N

N	PUT			GET		
	media	p50	p95	media	p50	p95
3	9.35	9.10	10.48	3.00	2.92	3.62
5	9.20	9.08	10.26	3.28	3.01	4.75
9	9.31	8.86	13.46	3.09	2.87	4.14
20	15.16	14.57	20.18	3.21	2.87	5.48

Il GET — una lettura locale sul Leader, teoricamente indipendente da N perché non coinvolge replicazione — resta piatto attorno ai 3ms su tutti i valori di N testati, esattamente come previsto dalla teoria. Il PUT resta piatto (circa 9.2–9.35ms) da $N=3$ a $N=9$, per poi salire nettamente a 15.16ms a $N=20$: coerente con Raft, dove il Leader deve attendere l'ack della maggioranza (11 su 20 follower a $N=20$) prima di poter committare, e la coda per l'ack più lento del gruppo di maggioranza cresce con il numero di follower.

B. Fault-tolerance: crash del leader

Lo strumento `experiments/fault-tolerance` avvia un cluster di 5 nodi, ne scopre il Leader corrente via la RPC `GetStatus`, lo uccide con `kill -9` per simulare un crash reale, e misura — via timestamp a precisione di microsecondo nei log dei nodi superstiti — due tempi: il tempo di *rielezione* (dal crash alla prima autoproclamazione di un nuovo Leader per un term successivo) e il tempo di *convergenza* (dal crash al momento in cui l'ultimo nodo superstite riconosce quel nuovo Leader), su 10 trial indipendenti (Tabella II).

Entrambi i tempi cadono dentro, o appena sopra, l'intervallo di election timeout configurato (150–300ms), come atteso. Il gap pressoché costante di circa 50ms tra rielezione e convergenza corrisponde esattamente all'heartbeat interval configurato: il nuovo Leader manda il proprio primo `AppendEntries` subito dopo essersi eletto, e i follower lo riconoscono quasi immediatamente. In tutti i 10 trial la convergenza è stata completa (i superstiti hanno tutti riconosciuto il nuovo Leader) e il servizio è sempre tornato operativo dopo il crash, verificato con una scrittura riuscita via proxy subito dopo la stabilizzazione del nuovo Leader.

Tabella II
TEMPO DI RIELEZIONE E CONVERGENZA DOPO IL CRASH DEL LEADER
(MS, N=5, 10 TRIAL)

	media	p50	p95	min	max
Rielezione	192.7	183.7	224.1	153.7	258.1
Convergenza	244.0	235.0	275.8	205.5	309.1

```
sleep 1
ps aux | grep -E "bin/consensus-node|bin/proxy" | grep -v grep | wc -l
docker images
0
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://%2Fvar%2Frun%2Fdocker.sock/_ping": dial unix /
var/run/docker.sock: connect: permission denied
sh-5.2$ sudo docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
sdcc-progetto-b5-kvstor-snapshot  latest             212c1411a9cd       7 days ago         24MB
sdcc-progetto-b5-kvstor-proxy      latest             d762f1cb71d6       7 days ago         24.1MB
sdcc-progetto-b5-kvstor-node2      latest             ee2b1b67b5ef       7 days ago         24.2MB
sdcc-progetto-b5-kvstor-node1      latest             f169ff3e83f8       7 days ago         24.2MB
sdcc-progetto-b5-kvstor-node3      latest             2ee8cde50e0a       7 days ago         24.2MB
hello-world              latest             e2ac70e7319a       5 months ago       10.1KB
sh-5.2$ sudo docker compose up -d
[+] up 5/5
✓ Container sdcc-progetto-b5-kvstor-node1-1   Started                                0.9s
✓ Container sdcc-progetto-b5-kvstor-node2-1   Started                                0.9s
✓ Container sdcc-progetto-b5-kvstor-node3-1   Started                                0.9s
✓ Container sdcc-progetto-b5-kvstor-snapshot-1 Started                                0.5s
✓ Container sdcc-progetto-b5-kvstor-proxy-1   Started                                0.5s
sh-5.2$ sleep 5
sudo docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED   STATUS    PORTS
sdcc-progetto-b5-kvstor-node1-1    sdcc-progetto-b5-kvstor-node1      "/usr/local/bin/serv..."            node1      7 days ago Up 29 seconds 0.0.0.0:50051->50051/tcp
sdcc-progetto-b5-kvstor-node2-1    sdcc-progetto-b5-kvstor-node2      "/usr/local/bin/serv..."            node2      7 days ago Up 29 seconds 0.0.0.0:50052->50052/tcp
sdcc-progetto-b5-kvstor-node3-1    sdcc-progetto-b5-kvstor-node3      "/usr/local/bin/serv..."            node3      7 days ago Up 29 seconds 0.0.0.0:50053->50053/tcp
sdcc-progetto-b5-kvstor-proxy-1    sdcc-progetto-b5-kvstor-proxy      "/usr/local/bin/serv..."            proxy      7 days ago Up 29 seconds 0.0.0.0:50060->50060/tcp
sdcc-progetto-b5-kvstor-snapshot-1 sdcc-progetto-b5-kvstor-snapshot  "/usr/local/bin/serv..."            snapshot   7 days ago Up 29 seconds
sh-5.2$ sudo docker compose logs node1 --tail 10
node1-1
node1-1 Peers not!
node1-1 - node1 & node1:50051
node1-1 - node2 & node2:50052
node1-1 - node3 & node3:50053
node1-1 Election timeout: [150ms, 300ms]
node1-1 Heartbeat interval: 50ms
node1-1 Log level: info
node1-1 2026/08/24 13:51:08 [node1] RaftService + KVStoreService in ascolto su 0.0.0.0:50051
node1-1 2026/08/24 13:51:08 [node1] avvio elezione per il term 4
node1-1 2026/08/24 13:51:08 [node1] eletto Leader per il term 4
sh-5.2$
```

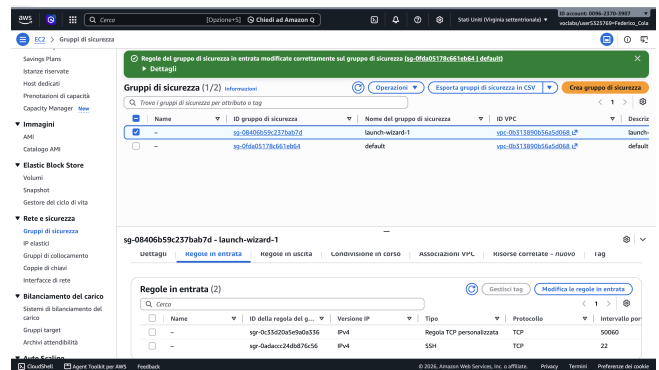
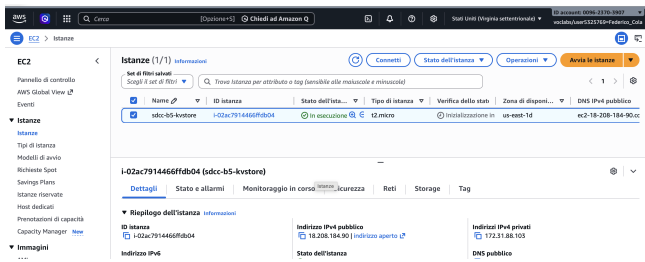


Figura 2. Evidenza del deployment su istanza AWS EC2 reale: build e avvio dei 5 container via Session Manager, con elezione del leader nei log (sopra); stato dell'istanza con IP pubblico (in basso a sinistra); regola del Security Group che apre la porta 50060 del proxy (in basso a destra).

VI. DISCUSSION

A. Interpretazione dei risultati sperimentali

I due esperimenti confermano, con dati, il comportamento atteso dell'algoritmo: il GET piatto su tutti gli N testati conferma che una lettura locale sul Leader è davvero indipendente dal numero di follower; il PUT piatto fino a $N=9$ e in salita a $N=20$ conferma che il costo della replica si manifesta solo quando il quorum di maggioranza diventa abbastanza numeroso da rendere statisticamente rilevante l'attesa dell'ack più lento. I tempi di rielezione e convergenza dell'esperimento di fault-tolerance cadono esattamente dentro l'intervallo di election timeout configurato, e il gap tra i due corrisponde all'heartbeat interval — nessuno dei due esperimenti ha richiesto di rivedere ipotesi sul funzionamento del sistema ed un suo successivo aggiornamento.

Una lezione metodologica più ampia, emersa nel percorso verso questi dati: la prima misura di scalabilità, fatta

sul portatile di sviluppo, produceva numeri rumorosi e non monotoni — al punto che persino il GET, che teoricamente non dovrebbe dipendere da N , mostrava le stesse oscillazioni del PUT. Questo è stato il segnale che il rumore di sistema (il portatile condiviso con l'ambiente di sviluppo stesso) dominava sull'effetto reale. Ripetere l'esperimento su un'istanza dedicata, pur con hardware più debole, ha eliminato quel rumore: la dedizione dell'ambiente di misura conta più della sua potenza di calcolo grezza.

B. La regola di sicurezza §5.4.2 osservata dal vivo

Durante il deployment su AWS EC2 (non durante gli esperimenti controllati di questa sezione, ma nel corso della verifica del sistema in un ambiente reale), un riavvio dell'istanza ha offerto una conferma pratica della regola discussa in Sezione II: una Get su una chiave scritta prima dello stop ha restituito `found=false`, non per perdita di dati ma perché `commitIndex` e `lastApplied` sono volatili

e si azzerano a ogni riavvio (Sezione IV). Il nuovo Leader, eletto in un nuovo term, non ha potuto far avanzare il proprio `commitIndex` oltre le entry di un term precedente finché non è avvenuta una nuova scrittura nel term corrente — a quel punto, `commitIndex` è avanzato in ordine di log oltre entrambe le entry in un colpo solo, e la vecchia chiave è ridiventata immediatamente visibile. Un episodio degno di nota che ha costituito una verifica del processo logico più sottile dell’algoritmo.

C. Limiti noti

Due scelte di scope, già motivate in Sezione IV, restano limiti espliciti del sistema così com’è: (1) nessun cambio di membership dinamico — un nodo può rientrare nel cluster dopo un crash, ma un nodo mai presente nella configurazione iniziale non può unirsi a runtime; (2) nessuna compattazione del log *sui consensus node stessi* — il servizio Snapshot & backup compatta correttamente lo storico in checkpoint esterni, ma non tronca il log persistito di ogni nodo, che quindi cresce indefinitamente con l’uso prolungato del sistema.

VII. CONCLUSIONS

Questo lavoro ha realizzato un key-value store distribuito, replicato e fortemente consistente, con un’implementazione di Raft costruita da zero (elezione del leader, replicazione del log, e le regole di sicurezza che governano l’avanzamento del commit) e verificata sia con unit test sia con processi reali comunicanti in rete. Tutti i requisiti architetturali della traccia sono soddisfatti: i tre tipi di nodo (consensus, client proxy, snapshot & backup), la comunicazione interamente su gRPC/Protocol Buffers, i due pattern architetturali richiesti (Service Discovery e Circuit Breaker), e un deployment containerizzato con Docker Compose verificato su una vera istanza AWS EC2, raggiungibile e funzionante dall’esterno.

I due esperimenti di valutazione richiesti — scalabilità e fault-tolerance — hanno prodotto dati puliti e coerenti con la teoria di Raft: latenza di lettura indipendente dal numero di nodi, latenza di scrittura che cresce solo quando il quorum di maggioranza diventa numeroso, e tempi di rielezione/convergenza dopo un crash del leader in linea con i parametri di timeout configurati. Il percorso verso questi risultati ha anche prodotto una lezione metodologica di per sé utile: misurare la latenza di un sistema distribuito richiede un ambiente di misura dedicato, non necessariamente potente.

Il sistema ha due limiti dichiarati, entrambi scelte di scope deliberate e non richieste dalla traccia: l’assenza di un meccanismo di cambio di membership dinamico (un nodo può ripresentarsi dopo un crash, ma non può unirsi ex novo a un cluster già in funzione), e l’assenza di compattazione del log *sui consensus node stessi* (il servizio Snapshot & backup compatta correttamente lo storico in checkpoint esterni, ma il log persistito di ogni nodo cresce indefinitamente). Entrambi rappresentano estensioni naturali per un lavoro futuro, così come l’esplorazione di cluster di dimensioni ancora maggiori per caratterizzare più a fondo la curva di scalabilità osservata tra $N=9$ e $N=20$.

RIFERIMENTI BIBLIOGRAFICI

- [1] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. Philadelphia, PA: USENIX Association, 2014, pp. 305–319.
- [2] L. Lamport, “Paxos made simple,” *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, 2001.